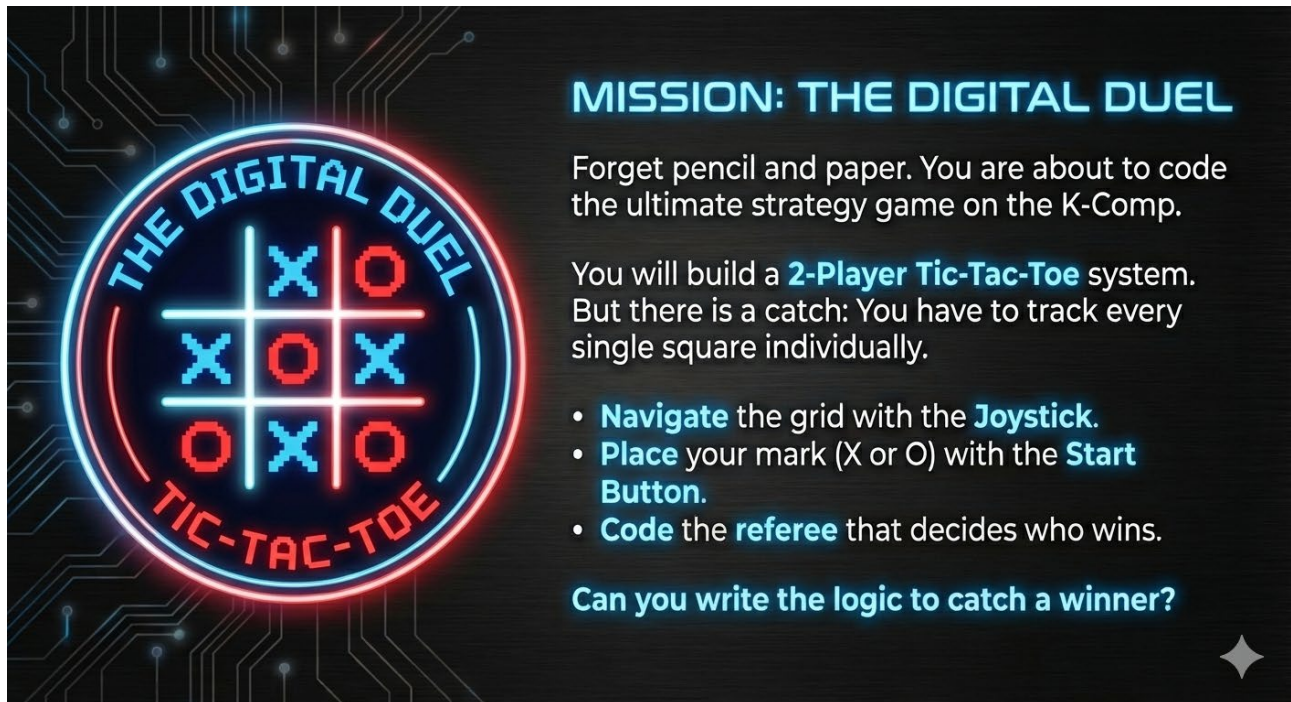




Welcome to the ***K-Comp Tic-Tac-Toe challenge!*** In this project, you will build a fully playable 2-player game. You will learn how to manage system memory, handle hardware interrupts, and draw to the OLED screen.

1. Preparations and Materials

You need the following components for this exercise:

- Your *K-Comp V2* board.
- 1 USB-B to USB-A cable.
- A Notebook having *Arduino IDE* installed.
- *The K-Comp V2 Board definition in the Arduino IDE.*¹

2. Game Programming Techniques

To build *Tic-Tac-Toe*, we will use some standard game development techniques tailored for embedded hardware reality.

a) The Hidden `main()` Function Standard C programs end when the code finishes. However, microcontrollers lack an operating system to return to and must run forever. To handle this, the Arduino framework hides the `main()` function and requires two specific alternatives:

- `setup()` : Runs exactly once on startup to initialize the hardware.
- `loop()` : Runs continuously in an infinite circle.

¹ https://raw.githubusercontent.com/andkuc/k-comp/refs/heads/main/board-definition/kcompV2/kcompV2_1_0_0_package_index.json

Standard C <pre> #define LENGTH uint8_t numbers[LENGTH] = {0}; int main(void) { setup(); numbers[0] = 5; while(1) { loop(); } return 0; // Never reached } </pre>	Arduino C <pre> #define LENGTH uint8_t numbers[LENGTH] = {0}; void setup() { // Runs one time at start numbers[0] = 5; } void loop() { // Repeated infinitely } </pre>
--	---

- b) Global Variables & Memory Limits** Desktop computers dynamically allocate memory, but the K-Comp features only 2 KB of SRAM. Dynamically creating and destroying memory on a microcontroller risks fragmentation, which will silently crash the board. Therefore, embedded C often uses global variables to strictly pre-allocate memory before the program even starts running. See the code example above.
- c) Strict Microcontroller Data Types** An `int` size varies by system (4 to 8 bytes on a desktop vs. 2 bytes on the 8-bit K-Comp). Guessing memory sizes is dangerous when you only have 2048 bytes of RAM. Instead, embedded developers control exactly how many bytes they consume using fixed-width data types:

Data Type	Description	Memory Size	Value Range
<code>bool</code>	boolean logic	1 byte	true (1) or false (0)
<code>uint8_t</code>	unsigned integer, 8-bit	1 byte	0 to 255
<code>int8_t</code>	signed integer, 8-bit	1 byte	-128 to 127
<code>uint16_t</code>	unsigned integer, 16-bit	2 bytes	0 to 65,535
<code>int16_t</code>	signed integer, 16-bit	2 bytes	-32,768 to 32,767

Best Practice: If you only need to track states like 0, 1, or 2, always use `uint8_t`. Using a standard `int` would waste 100% more memory than necessary! Likewise, for simple on/off or true/false flags, always stick to `bool`.

To build *Tic-Tac-Toe*, we will use a few simple game development techniques.

Computerarchitekturen und Betriebssysteme		
CABS4A GU, KS, KU, LN, MF, WB	Programming K-Comp – Tic Tac Toe	

Technique	Description
<i>The Game Loop</i>	Instead of running top-to-bottom once, games run in an infinite circle. In Arduino C, this is the <code>loop()</code> function. Hundreds of times per second, the code does three things in order: • <u>Input</u> (read sensors) • <u>Update</u> (calculate logic) • <u>Render</u> (draw to screen).
<i>State Machines</i>	A game can't be in "Menu Mode" and "Game Over Mode" at the same time. We use a <u>game state variable</u> to act like a switch, telling the code exactly which set of rules to follow at any given moment. This is used to logically map different game states using simple <code>if</code>-statements.
<i>Memory Management</i>	Instead of "normal" variables, we will allocate a specific chunk of memory (=array) to represent our 3x3 grid.
<i>Hardware Polling & Callbacks</i>	We will use <u>event-driven programming</u> to listen for button clicks without pausing the game.
<i>Non-Blocking Timers</i>	If we used a basic wait command, the whole game would freeze. Instead, we use a system stopwatch (<code>millis()</code>) to check how much time has passed, ensuring the joystick doesn't move too fast while keeping the code running smoothly.

3. Pre-Flight Check: Exploring the Hardware Examples

Before you write your game, you must understand how the individual components function. The  `/examples` folder contains a curated folder of sample projects, ranging from basic hardware diagnostics to fully realized interactive systems.

Steps:

1. Open the examples for the **OLED**, the **oled grid engine**, the **joystick**, the **buttons**, the **LED**, and the **buzzer**.
2. Try out the test programs by uploading them to your *K-Comp* board. Observe how the hardware responds and review how the API commands are structured in the code.

Computerarchitekturen und Betriebssysteme		
CABS4A GU, KS, KU, LN, MF, WB	Programming K-Comp – Tic Tac Toe	

4. Iteration 1 - The Empty Board

Goal: Compile your first program, initialize the screen, and see the empty 3×3 grid.

Steps:

1. Open your IDE create a new sketch and select K-Comp V2 as device. Include the headers for the **OLED** display and the **grid engine** at the top.

Hint: Check the documentation in  /docs for the exact filenames.

2. Create a specific memory allocation for the grid using `uint8_t`.

Hint: Because embedded C relies on pre-allocated memory, you must declare this as a *global array* outside of your functions so the space is permanently reserved. Look at the `kcomp_oled_grid` examples to see how the `GRID_MEM_SIZE` macro is used.

3. In `setup()`, initialize the display and the grid.
4. In `loop()`, clear the display, render the grid, and update the display.

Upload your code. You should see a blank 3×3 grid on the OLED screen!

Save your program on your computer having the name **`TicTacToe_<yourname>_Iteration1_Setup.ino`**.

5. Iteration 2 - The Cursor & Joystick Navigation

Goal: Add player input so you can move a cursor around the grid.

Steps:

1. Include the **joystick** header.
2. Create a `drawCursor` function to show the player which square they are hovering over.
Hint: Open the API documentation in  /docs. Look up the **OLED** documentation for drawing commands like `drawRect`.
3. In `setup()`, initialize the joystick, attach your cursor render function, and enable the cursor.
4. In `loop()`, read the **joystick** using `getJoystickXValue()` and `getJoystickYValue()`. Call `moveCursor()` immediately based on the analog thresholds (e.g., `> 250` or `< -250`).

Upload your code. Push the joystick—you should now be able to navigate the cursor across the grid seamlessly!

Save your program on your computer having the name **`TicTacToe_<yourname>_Iteration2_CursorJoystick.ino`**.

Computerarchitekturen und Betriebssysteme		
CABS4A GU, KS, KU, LN, MF, WB	Programming K-Comp – Tic Tac Toe	

6. Iteration 3 - Placing Pieces & Hardware Feedback

Goal: Allow players to press the **SELECT** button to place a game piece (X) and hear a confirmation beep.

Steps:

1. Include the headers for the **SELECT** button, **LED**, and **Buzzer**.
2. Create a `drawPieces` function. For now, if the cell state is 1, draw an 'X' using lines. Have a look at the grid example again and attach this function to the grid engine in your `setup()`.
3. Create the `onSelectClicked` function. Check if the current grid cell is empty (0). If it is, write a 1 to the grid cell, flash the **LED**, and beep the **buzzer**.
4. Update `setup()` to initialize the **buttons**, **LED**, and **Buzzer**, and attach your click event.
5. Update `loop()` to poll the **SELECT** button and call `updateMelody()` to keep the audio engine running.

Upload your code. Navigate to a square and click SELECT. You should hear a beep, see a flash, and an 'X' will appear!

Save your program on your computer having the name **TicTacToe_<yourname>_Iteration3_PlacementFeedback.ino**.

7. Iteration 4 - Two-Player Turns

Goal: Make it a real game by switching between Player 1 (X) and Player 2 (O).

Steps:

1. Create a global `uint8_t currentPlayer` variable and set it to 1.
2. Update your `drawPieces` function: if the state is 2, draw an 'O' using a circle.
3. Update `onSelectClicked`: Instead of always writing a 1, write the value of `currentPlayer` to the cell. Then, use an `if/else` statement to swap `currentPlayer` from 1 to 2 (or 2 to 1).
4. In your `loop()`'s render phase, use `setCursor()` and `printText()` to draw "P1 Turn" or "P2 Turn" on the side of the screen so players know who is up.

Upload your code. You can now play a full, alternating game of Tic-Tac-Toe with a friend! (Though the game doesn't know who won yet).

Computerarchitekturen und Betriebssysteme		
CABS4A GU, KS, KU, LN, MF, WB	Programming K-Comp – Tic Tac Toe	

Save your program on your computer having the name ***TicTacToe_<yourname>_Iteration4_TwoPlayer.ino***.

8. Iteration 5 - The State Machine (Menu & Reset)

Goal: A real game has a Start Menu, a Playing phase, and a Game Over phase. We will implement a Finite State Machine (FSM).

Steps:

1. Include the header for the **START** button.
2. Create a global `uint8_t gameState` variable. (0 = Menu, 1 = Playing, 2 = Game Over).
3. Create an `onStartClicked` function. If the state is Menu or Game Over, wipe the grid clean, set `currentPlayer` to 1, and change the `gameState` to 1 (Playing).
4. Restructure your `loop()`: Use `if` statements so that the joystick is only read, and the grid is only rendered, IF the `gameState` is 1. If it is 0, just render a "PRESS START" text.

Upload your code. You should boot into a Menu, press START to play, and press START anytime to wipe the board in a panic!

Save your program on your computer having the name ***TicTacToe_<yourname>_Step5_FSM.ino***.

9. Iteration 6 - The Win Condition & Flash Feedback

Goal: The game needs to know when someone gets three-in-a-row.

Steps:

1. Create a new function called `checkWin()`. Inside it, use `if` statements to check the rows, columns, and diagonals of the global `gameMemory` array.
2. If there are three 1s or three 2s in a row, change the `gameState` to 2 (Game Over).
3. Call `checkWin()` at the very end of your `onSelectClicked` function.
4. **Flash Feedback:** Use the `invertDisplay()` command (check the API docs) to briefly invert the screen colors to startle the player when someone successfully wins.
5. Update your `loop()` so that if `gameState` is 2, it prints "WINNER!" on the screen.

Upload your code. Play a match. When someone wins, the screen will flash, and the game will freeze on the Game Over state until you press START!

Computerarchitekturen und Betriebssysteme		
CABS4A GU, KS, KU, LN, MF, WB	Programming K-Comp – Tic Tac Toe	

Save your program on your computer having the name ***TicTacToe_<yourname>_Step6_WinTheGame.ino***.

10. Iteration 7 - The Tie Game Check

Goal: What happens if the board is full but nobody won?

Steps:

1. Inside your `checkWin()` function (or a new `checkTie()` function), add logic to loop through the 9 grid cells.
2. If the board is full (no 0 spaces left) and no one has won, play a sad, low-frequency buzzer tone.
3. Change the `gameState` to 2 (Game Over) but trigger a special "TIE" text flag so the `loop()` prints "TIE GAME" instead of "WINNER!".

Upload your code. Intentionally play a tie game. Observe the Tie state.

Save your program on your computer having the name ***TicTacToe_<yourname>_Step7_Tie.ino***.

11. Iteration 8 - High Score Tracking

Goal: Add a competitive edge by keeping track of how many times Player 1 and Player 2 have won across multiple rounds.

Steps:

1. At the top of your program, create two new global `uint8_t` variables (e.g., `scoreP1` and `scoreP2`) and initialize them to 0.
2. Update your `checkWin()` logic. If Player 1 wins, increment `scoreP1` by 1. If Player 2 wins, increment `scoreP2` by 1.
3. In your Main Loop's render phase (during the Menu or Playing states), draw the current scores constantly at the bottom or sides of the **OLED** screen, so players always know who is in the lead.

Upload your complete code to the K-Comp. Congratulations, you have engineered a fully structurally sound C game!

Save your program on your computer having the name ***TicTacToe_<yourname>_Step8_HighScore.ino***.